AimMaster Project Report

Real-Time Multiplayer Hand-Gesture Shooting Game

1. Project Overview

1.1 Problem Statement

AimMaster addresses a simple but real problem: playing a multiplayer aim-and-shoot game using nothing but your hand and a webcam. Most browser games rely on a mouse, keyboard, or a gamepad, which locks out anyone who wants a more physical, gesture-driven experience. In a classroom or lab setting, students often need a project that combines several things at once — real-time networking, a game loop, concurrency, authentication, and a live frontend — but building all of those pieces from scratch in one go is overwhelming. The people affected are players (who get a novel way to play) and students (who get a working, well-structured codebase they can learn from). It matters because it shows how a real-time multiplayer game can be built with plain, standard tools, without any heavyweight game engine.

1.2 Existing Gaps

Most existing approaches fall short in a few ways. Simple browser games are usually single-player only, so they never have to solve the hard problems of real-time sync between multiple people. Larger multiplayer games tend to depend on a game engine or a framework, which hides the interesting parts (the game loop, hit resolution, WebSocket broadcasts) behind abstractions, so you never really learn how they work. Tutorials that do cover WebSockets often stop at a chat app, which never touches game-specific concerns like an authoritative server, a tick loop, or a race between two players firing at the same target at the same instant. What was missing was a small, self-contained project that puts all of these together and keeps each responsibility in its own clearly separated class.

1.3 Solution

AimMaster is a Spring Boot backend with a plain HTML5 Canvas + vanilla JS frontend. The server is the single source of truth: it owns the game loop (a dedicated thread per room), moves the targets, decides every hit, keeps the live leaderboard in memory, and broadcasts the arena state to everyone in the room about ten times a second. The browser only ever draws what the server confirms, and when a player "fires," it sends a claim ("I think I hit target X") that the server verifies atomically. Players register and log in with JWTs, create or join rooms by code, and use MediaPipe hand tracking to aim with their index finger and fire by folding their thumb. Spectators can watch any match live without taking a player slot. The whole thing runs on an embedded server with an in-memory H2 database and no external image or CDN dependencies for the game visuals.

2. Core Feature List

- **Room creation and join-by-code:** a host creates a room with a difficulty, duration, and player cap; other players join with a six-character code.
- **JWT authentication:** register and log in, with tokens stored in sessionStorage so a page refresh keeps you logged in.
- **Real-time arena broadcasts:** the server pushes target positions and elapsed time to every player and spectator in the room about ten times a second.
- **Hand-gesture aiming and firing:** MediaPipe tracks the index fingertip as a crosshair and the thumb-fold as a discrete fire trigger (edge-triggered, so holding the thumb folded does not spam shots).
- **Authoritative hit resolution:** the server atomically decides whether a shot lands, so two players firing at the same target at the same instant can never both score.
- **Live leaderboard:** every room member is pre-populated at score zero when the match starts, and scores update in real time for everyone.
- **Match lifecycle:** the host starts and stops the match, or it auto-ends after the configured duration.
- **Match history and stats:** after a match ends, each player's score, fire attempts, and accuracy are saved and shown in their personal history.
- **Disconnect handling:** if a player drops, the others see a "left the room" notice and the match keeps going; if everyone leaves, the match stops cleanly with no thread leak.
- **Reconnect handling:** a dropped connection retries every few seconds and repaints targets, time, and scores from a REST "catch me up" endpoint without a page reload.
- **Spectator mode:** anyone can watch a live match (even a full one) with no camera, no firing, and no effect on match lifecycle.
- **Admin dashboard:** an allowlisted admin user sees every running match, per-room player/spectator counts, and JVM thread count, live.
- **Bullet visuals:** a glowing tracer flies from the wrist to the target (homing while it moves) and an impact ring pops on landing, with the "+1" appearing only when the bullet actually lands.
- **Image-sprite targets and scrolling background:** targets are drawn as inline SVG sprites, they spawn at the right edge and drift left, and a themed background image scrolls continuously behind the arena.
- **"User joined" notification:** when a player joins a room, everyone currently in it sees a brief banner naming who just arrived.

3. System Design & Diagrams

The system is split into a Spring Boot backend and a vanilla JS frontend, talking over two channels: REST for "catch me up" requests (login, room CRUD, live-state snapshots, history) and STOMP-over-WebSocket for the live broadcasts (arena state, hit results, join/leave notices). This split is deliberate: REST handles requests that can wait a beat, while the WebSocket keeps the screen updating smoothly afterward.

On the server, each room that is mid-match gets its own GameSession, which owns its own single-thread ScheduledExecutorService. That thread runs the game loop: move targets, despawn off-screen ones, maybe spawn new ones, and broadcast the arena state — about ten times a second. Because only that one thread ever writes to the arena, no synchronization is needed on target positions. GameSessionManager keeps a map of room code to GameSession and is the only thing that creates or removes sessions, so two rooms can never interfere with each other. HitResolver atomically kills a target (compare-and-set), and only the thread that wins the race awards the point. Leaderboard keeps scores in a ConcurrentHashMap of AtomicIntegers so simultaneous hits never overwrite each other. When a match ends (host stops it, the duration timer fires, or the room empties), GameSession.stop() runs exactly once — guarded by a compare-and-set on an "active" flag — which releases the thread and hands the match to MatchRecordService for persistence.

Component flow (text diagram):

```
Browser (Canvas + MediaPipe)
   |  STOMP: /app/room/{code}/fire  (fire claim)
   v
RoomWebSocketController  ->  HitResolver  ->  Leaderboard.addPoint
   |                                      (only the race winner scores)
   v
RoomBroadcaster  ->  /topic/room/{code}  +  /topic/room/{code}/spectators
   ^                                              ^
   |                                              |
GameSession (one dedicated thread per room)       |
   gameEngine.tick(): move, despawn, spawn, broadcast arena state
   |
   on end -> GameSessionManager -> MatchRecordService (persist to H2)
   |
REST (anytime): GET /api/rooms/{code}/live  ->  catch-up snapshot
```

Figure 1: *Request and broadcast flow between browser and server.*

4. Tech Stack

Frontend: Plain HTML5 Canvas + vanilla JavaScript (no frameworks, no build step). MediaPipe Hands for hand-landmark detection; SockJS + STOMP.js for WebSocket; inline SVG data URIs for target sprites and the background image (no external image hosting).

Backend: Java 17, Spring Boot 3.2 (Spring Web, Spring Security, Spring Data JPA, Spring WebSocket / STOMP).

Database: H2 in-memory database (JPA entities for Player, Room, and MatchRecord; live game state like targets and the leaderboard stays in memory by design).

Tools / Libraries / Services: Maven 3.9 for build; jjwt 0.11.5 for JWT creation and verification; ngrok used during testing to expose localhost over HTTPS so the camera (getUserMedia) works from other devices on the network.

5. Challenges You Faced & How You Solved Them

Challenge 1: Two players firing at the same target at the same instant could both score, because a plain "check if alive, then kill" has a race window between two threads.

> **Solution:** Solution: Target.alive is an AtomicBoolean, and dying goes through tryMarkDead(), which atomically flips true to false and reports whether THIS call was the winner. HitResolver only awards the point if tryMarkDead() returned true, so exactly one shot ever scores per target. A dedicated test (HitResolverConcurrencyTest) fires two threads at the same target at the same instant and asserts exactly one hit.

Challenge 2: scheduleAtFixedRate silently stops scheduling all future ticks forever if a tick throws an uncaught exception — the room would quietly freeze with no error.

> **Solution:** Solution: GameSession.tick() wraps the whole tick in a try/catch that logs the error, so a bug in one tick can never kill the whole match.

Challenge 3: stop() can be reached two ways at once — a host manually ending the match, or the duration timer firing on its own — and match persistence must run exactly once.

> **Solution:** Solution: stop() is guarded by active.compareAndSet(true, false), so only the first call actually stops the loop and fires the onEnded callback. Everything downstream (removing the session from the map, persisting records) happens from that one spot.

Challenge 4: A player who joined after the host clicked "start" could rack up a real score on the live leaderboard and then vanish from match history, because the participant snapshot only knew who was there at the start.

> **Solution:** Solution: GameSession.getAllInvolvedUsernames() unions the start-time participants with anyone who fired or scored mid-match, and that is what MatchRecordService persists. Joining mid-match now also adds the player to the live leaderboard at score zero (via RoomService calling GameSession.addPlayerToLeaderboard), so they show up immediately.

Challenge 5: The camera (getUserMedia) is blocked by browsers on plain HTTP for any origin other than localhost, so a friend on the LAN could not enable the camera to play.

> **Solution:** Solution: Expose the local server over HTTPS using an ngrok tunnel (ngrok http 8080), which gives a public https:// URL. Since browsers treat HTTPS as a secure context, the camera permission prompt works from any device.

Challenge 6: The frontend originally drew the live webcam feed and a scrolling background, but the desired look was a continuously panning background image with the camera feed removed entirely.

> **Solution:** Solution: renderFrame() now always draws the background image, panning it right-to-left with a wrapping offset (drawing the overflow on the opposite edge so it loops seamlessly), and the camera-feed drawing path was removed. MediaPipe still uses the camera for hand tracking, so gesture aiming keeps working.

6. Screenshots of the Main Pages & Processes

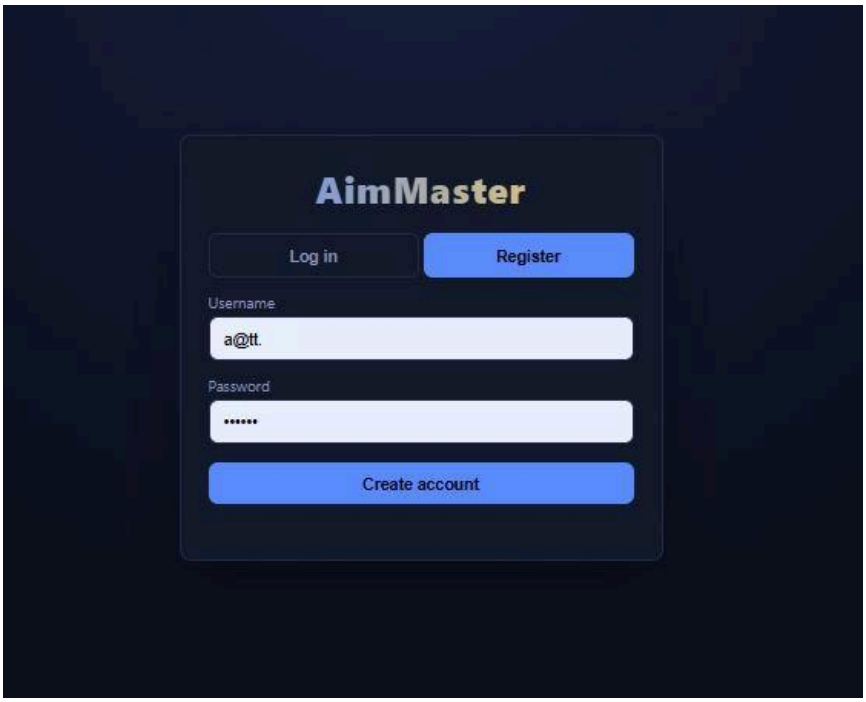


Figure 2: Register Page. The Register tab of the entry screen, where a new player enters a username and password and clicks "Create account".



Figure 3: Login Page. The Log in tab, where an existing player signs in with a username and password to receive a JWT session.

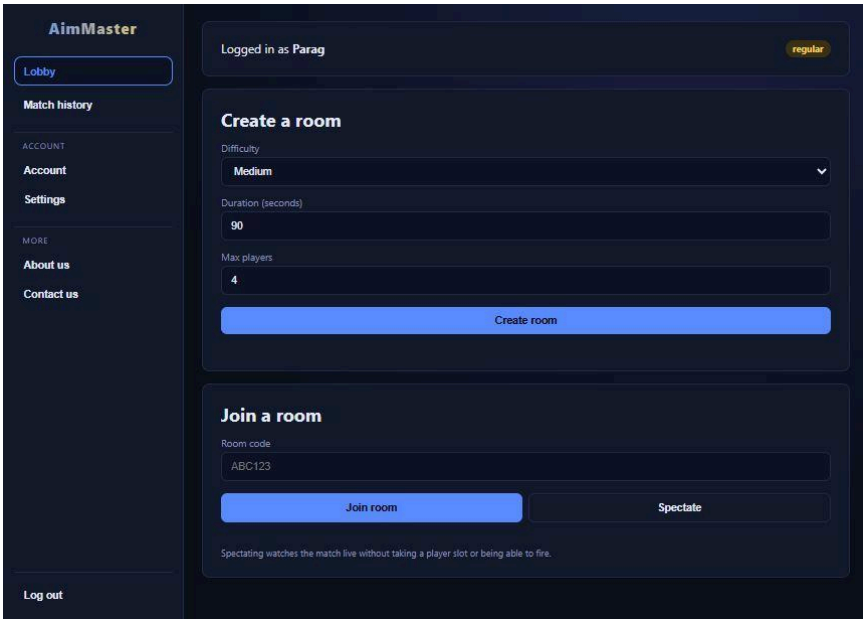


Figure 4: Lobby Page. The main lobby: a host creates a room by choosing difficulty, duration and max players, and other users join with a room code or spectate.

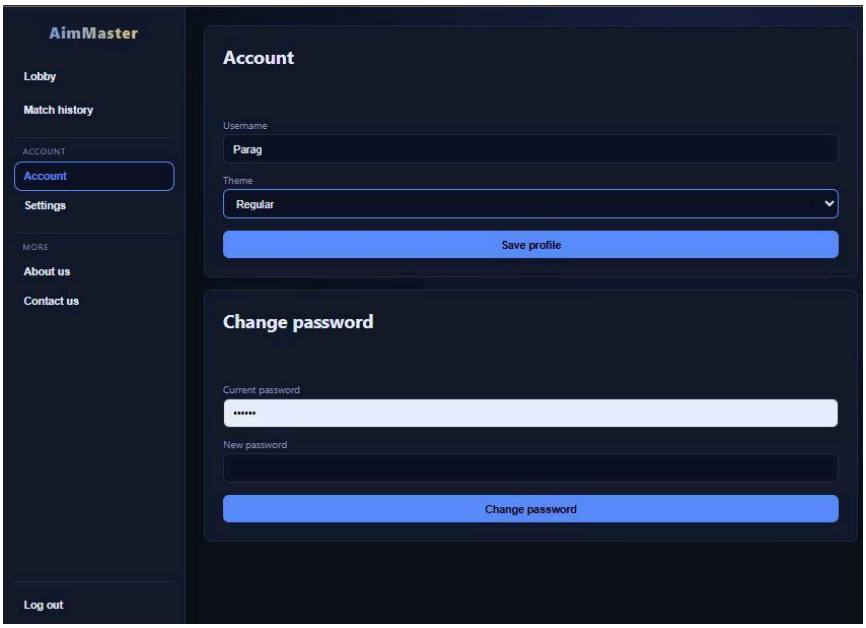


Figure 5: Account Page. The Account page, where the player can update the profile (username and theme) and change the password.

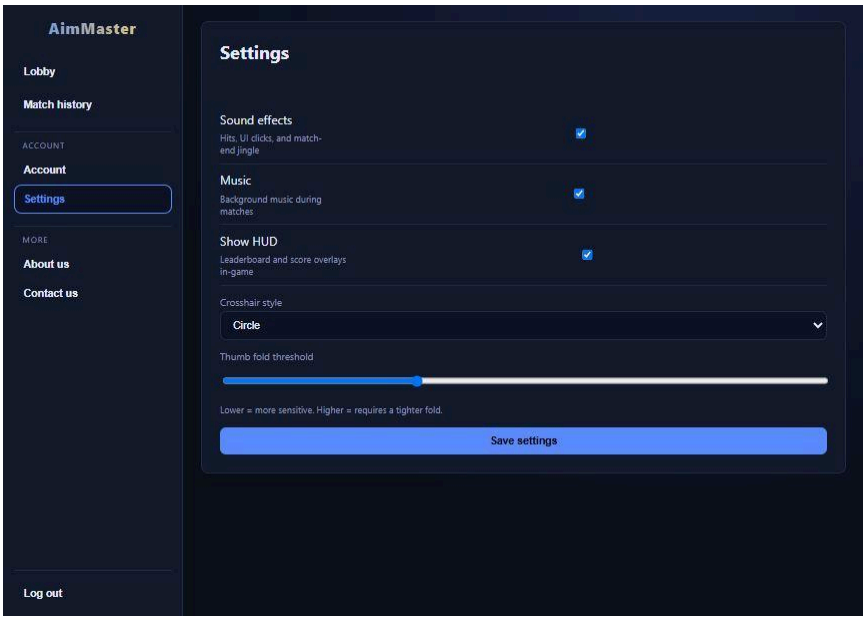


Figure 6: Settings Page. The Settings page with toggles for sound effects, music and HUD, plus crosshair style and thumb-fold sensitivity for gesture firing.

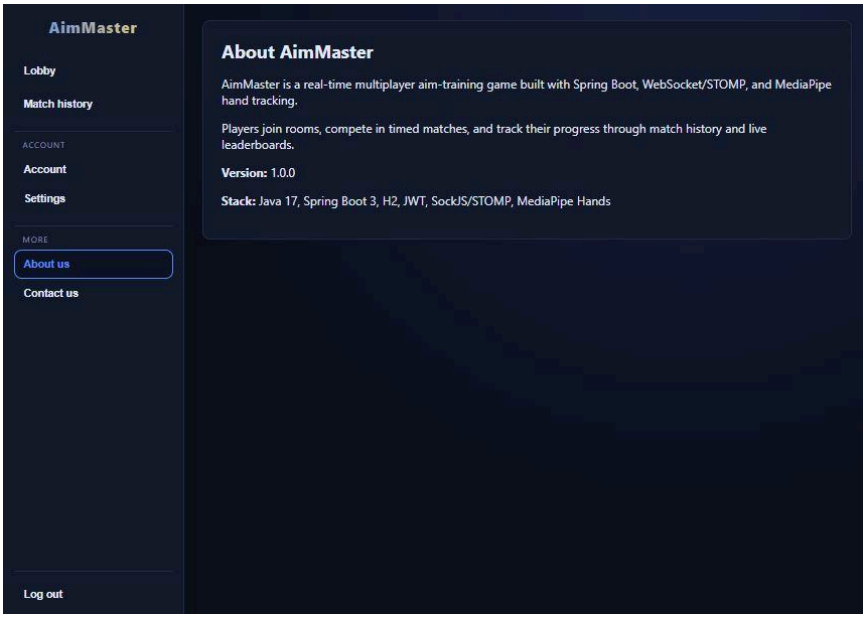


Figure 7: About Us Page. The About page, giving a short description of AimMaster, its version and the technology stack.



Figure 8: Contact Us Page. The Contact page, where a user can send a bug report or feature request through a simple subject and message form.

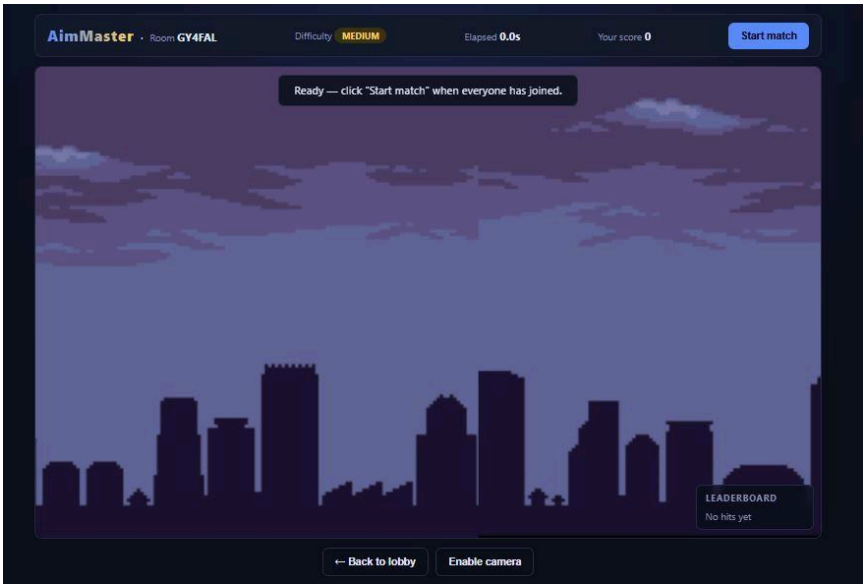


Figure 9: Game Room Before the Match. The room screen after creation, showing the room code, difficulty, timer, score and the host's "Start match" button, with an empty leaderboard.

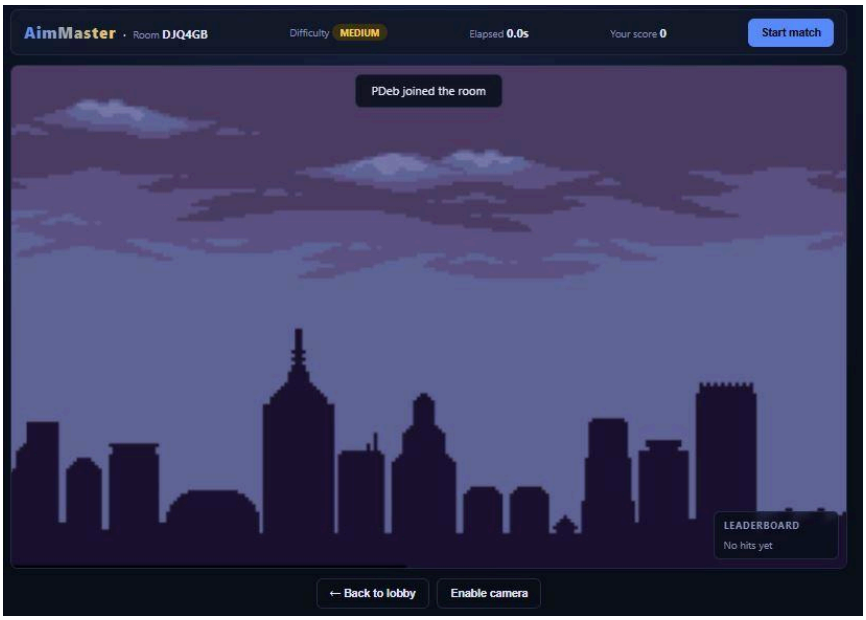


Figure 10: User Joined Notification. A banner broadcast to everyone in the room when another player (PDeb) joins before the match starts.

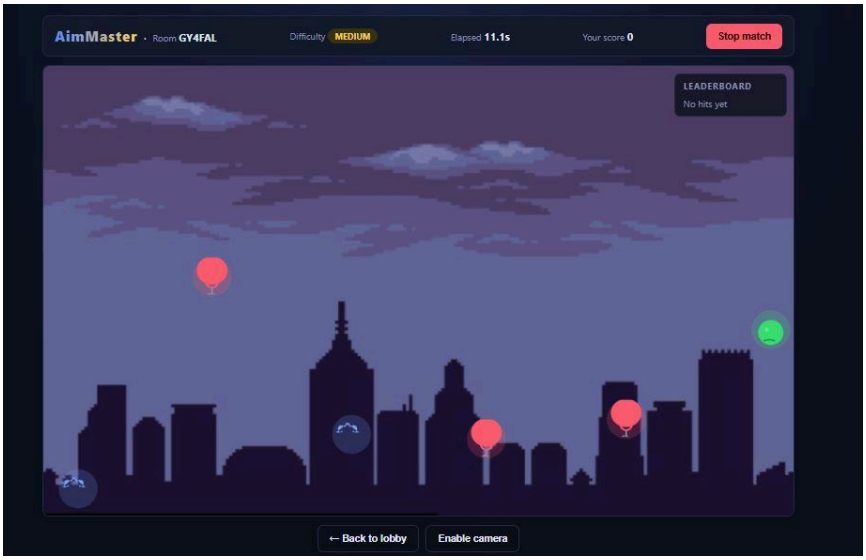


Figure 11: Live Match with Targets. A running match where balloon and smiley targets drift across the arena while the elapsed time and "Stop match" control are shown.

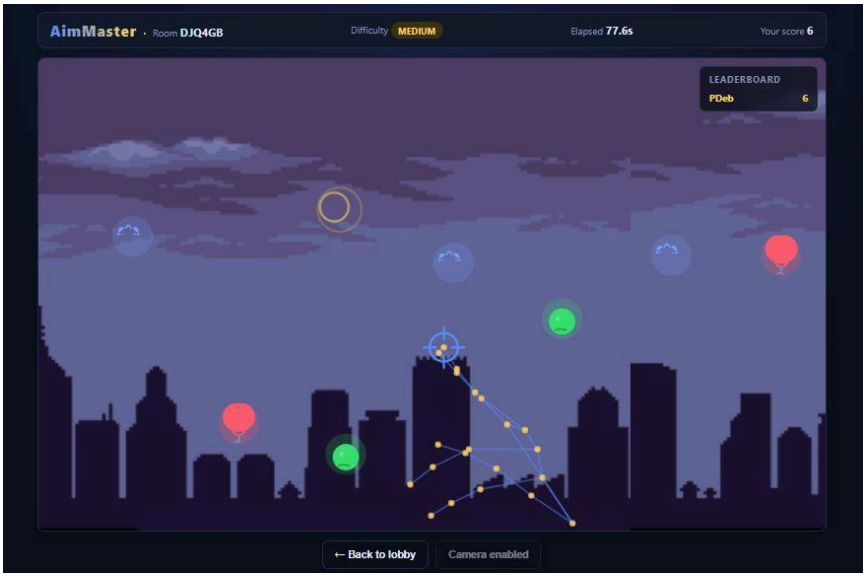


Figure 12: Hand-Gesture Aiming and Shooting. MediaPipe hand tracking draws the hand landmarks, the index fingertip acts as the crosshair, and an impact ring appears where a shot landed.

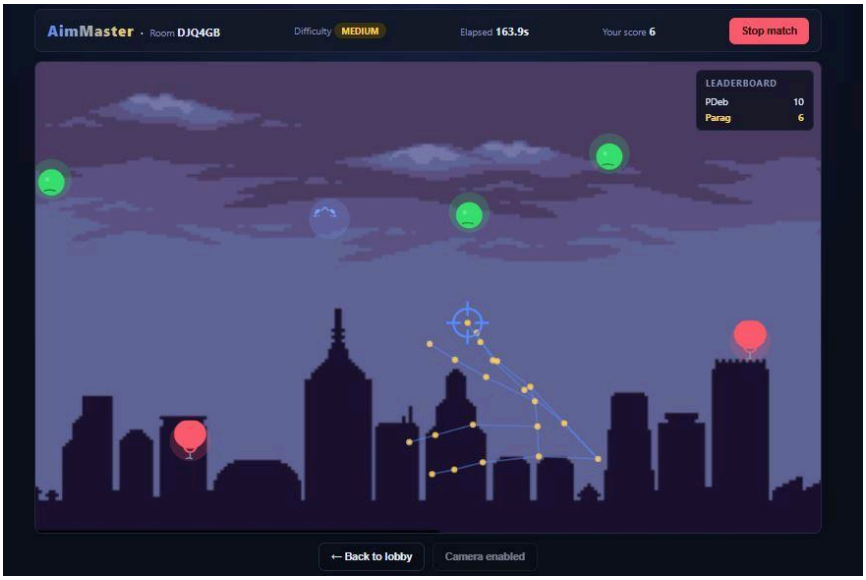


Figure 13: Live Leaderboard During a Match. The in-game leaderboard updating in real time with both players' scores (PDeb and Parag) while the match is in progress.

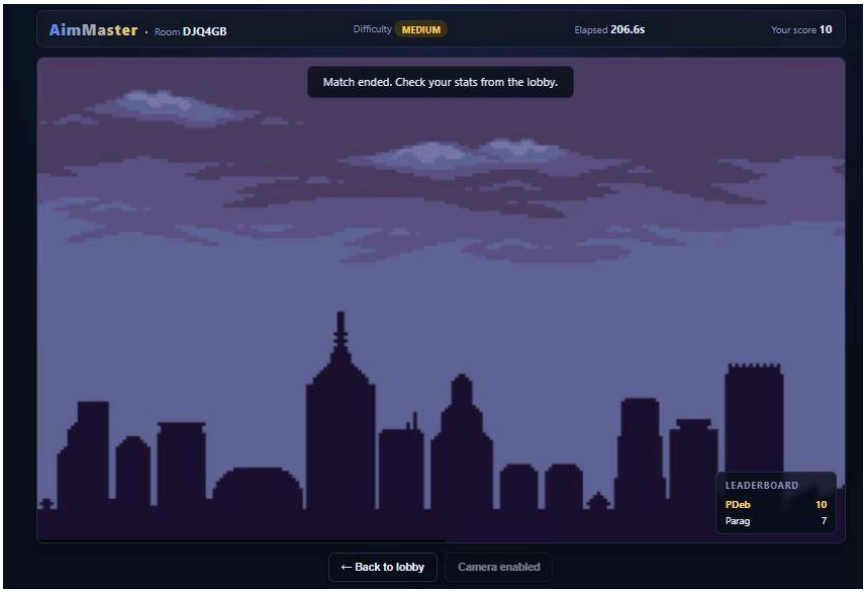


Figure 14: Match Ended Screen. The state after the host leaves or the match ends, with a "Match ended" notice and the final leaderboard scores.

6. Github Link:

Repository Link : [Link]

8. Video Link:

8.1 Video 1: Link

8.2 Video 2: Link